King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 11
Hashing

Term 252

Reading Assignment

- "Data Structures and Algorithms in Java", 4th Edition, Adam Drozdek, Cengage Learning, ISBN 978-981-4392-78-5
  - Chapter 10
    - Section 10.1: Hash Functions
    - Section 10.2: Collision Resolution (Except 10.2.3: Bucket Addressing)
    - Section 10.3: Deletion

Introduction to Hashing & Hashing Techniques

- Review of Searching Techniques
- Introduction to Hashing
- Hash Tables
  - Types of Hashing
- Hash Functions
  - Applications of Hash Tables
  - Problems for which Hash Tables are not suitable
- Collision Resolution Techniques
  - Separate Chaining
  - Open Addressing
    - Linear Probing
    - Quadratic Probing
    - Double Hashing
    - Rehashing
  - Algorithms for insertion, searching, and deletion in Open Addressing
- Separate Chaining versus Open-addressing

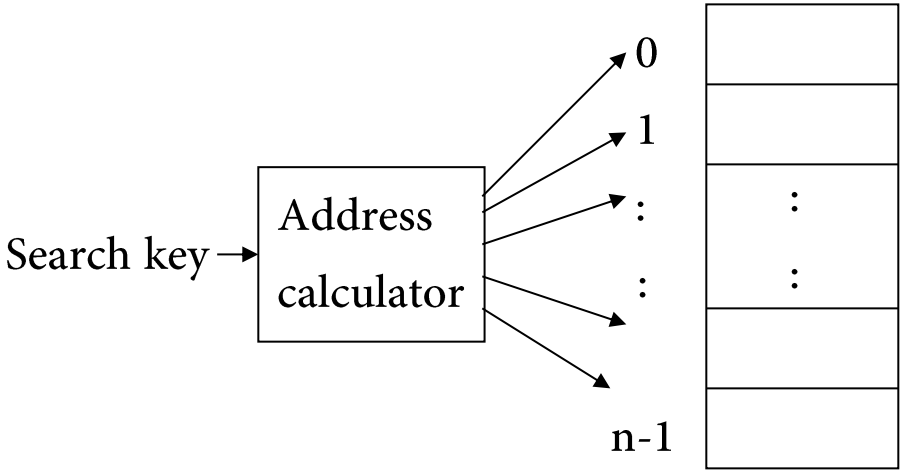
Review of Searching Techniques

- The search problem: find items with keys matching a given search key

- The efficiency of these search strategies depends on the number of items in the data structure being searched, i.e. input size.

- Search methods with efficiency independent of data size would be better.
  - Can we jump directly to the place where the element should be?

Search	Complexity
Linear search	O(n)
Binary Search	O(log n)
BST	O(log n) or O(n)
AVL tress	O(log n)



Review of searching Techniques...

- At KFUPM, we have about 10,000 students. **Student IDs** are 9 digits.
- The id is used as a search key.
- To store 10,000 students' records (each with a 9-digit ID):

Implementation	Complexity
A linked list	
A height balanced tree (AVL)	
An array of size 10^9	

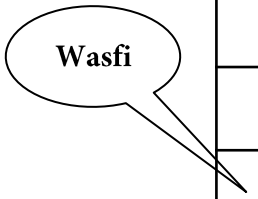
```
Student {
    long id;
    String name;
    double GPA;
    ...
}
```

- Is there some way that we could get O(1) access without ~~wasting~~ a lot of space?
  - The answer is hashing. map these KEYS (9-digit IDs) into a smaller array

Example 1: Illustrating Hashing

- We have 8 records. We want to store them in an array of size 13.
- Use the function f(r) = r.id % 13

Name	ID	f(r) = id % 13
Al-Otaibi, Ziyad	985926	6
Al-Turki, Musab Ahmad Bakeer	970876	10
Al-Saegh, Radha Mahdi	980962	8
Al-Shahrani, Adel Saad	986074	11
Al-Awami, Louai Adnan Muhammad	970728	5
Al-Amer, Yousuf Jauwad	994593	2
Al-Helal, Husain Ali AbdulMohsen	996321	1
Al-Khatib, Wasfi Ghassan	863523	11



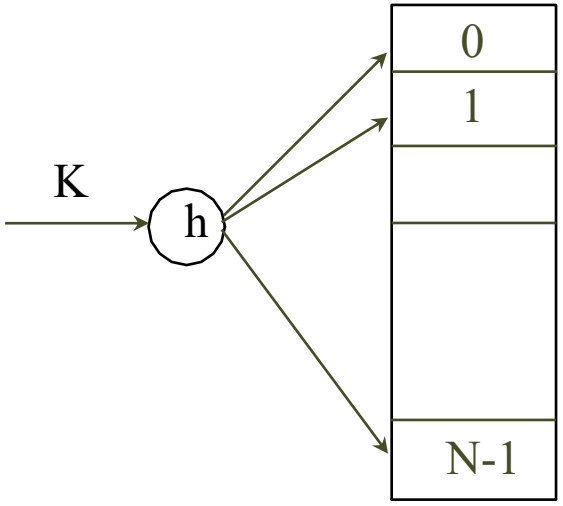
0	
1	Husain
2	Yousuf
3	
4	
5	Louai
6	Ziyad
7	
8	Radha
9	
10	Musab
11	Adel
12	
13	

Hash Functions

- A hash function, h, is a function which transforms a key from a set, K, into an index in a table of size n:

$$h : K \rightarrow \{0, 1, ..., n-2, n-1\}$$

Hash Function Key Memory Address



  - A key can be a number, a string, a record etc.
  - The size of the set of keys, |K|, to be relatively very large.
  - Ideally, we would like to have a 1-1 map
    - It is possible for different keys to hash to the same array location.
    - This situation is called collision, and the colliding keys are called synonyms.

Hash Functions

- Desired Properties of Hash Functions: a good hash function should:
  - Minimize collisions
  - Be easy and quick to compute
  - Distribute key values evenly in the hash table
  - Use all the information provided in the key
  - Deterministic: same input ⟶ same output

- Common Hashing Functions
  - Division Remainder (using the table size as the divisor)
  - Folding
  - Mid-Square
  - Truncation or Digit/ Character Extraction
  - Radix Conversion
  - Use of a Random-Number Generator

Common Hashing Functions

(1) Division Remainder: h(key) = key mod table size

key = 25936715; table size = 10 ⟶ h(key) = 25936715 % 10 = 5

Table size better if it is prime
Sequential keys may collide

(2) Folding: Break the key into equal parts and add them

Key = 123456⟶split 12|34|56 ⟶add 12+34+56 =102⟶ h(key)=102 % 10 =2

Useful for large keys;
Fast and simple especially with bit patterns;

(3) Mid-Square: Square the key and extract middle digits

key = 3121 ⟶ Key2 = 9740641 ⟶ Take the middle 406

Good if the keys do not contain a lot of leading or trailing zeros.

(4) Truncation: Pick specific digits of the key

key = 58263914 ⟶ Choose digits **3rd and 5th** ⟶ "2" and "3" ⟶ h(key)= **23**

Useful for large keys;
Fast and simple especially with bit patterns;

(5) Radix Conversion: Transforms the key to a new base

Key = 55354_{10} ⟶ 38652_{11} ⟶ h(key)=38652 % 1000 = 652

Good if the keys do not contain a lot of leading or trailing zeros.

Types of Hashing

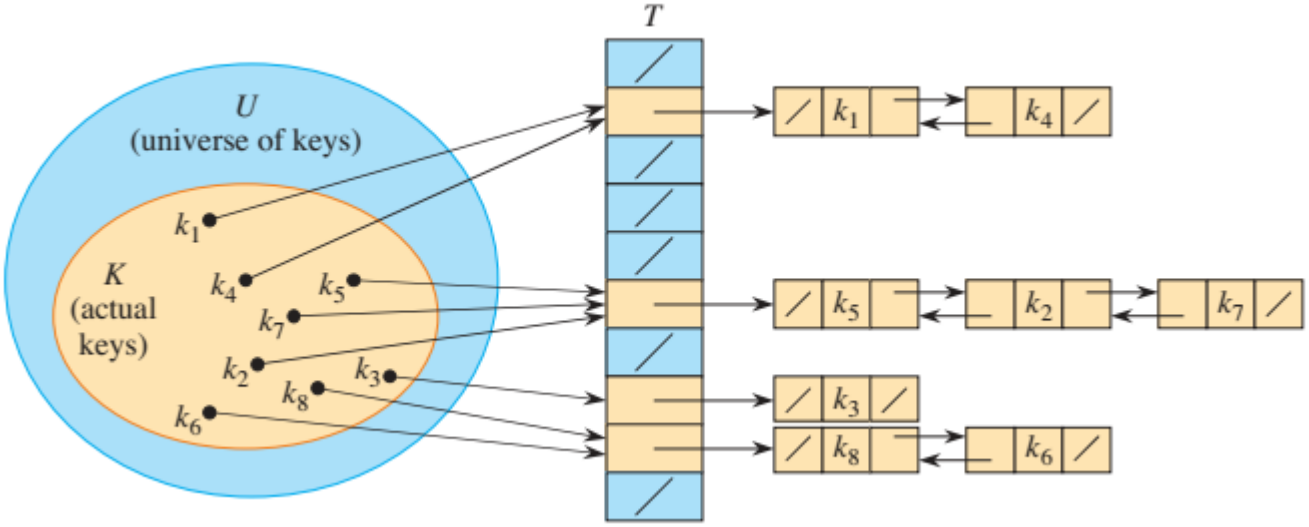
- by Table Size:
  - Static hashing:
    - keys are mapped to a fixed set of locations.
  - Dynamic hashing:
    - The hash table can grow to handle more items.
    - The associated hash function must change as the table grows.

- By handling collision:
  - Separate Chaining:
  - Open Addressing: Array-based implementation.
    - (i) Linear probing (linear search)
    - (ii) Quadratic probing (nonlinear search)
    - (iii) Double hashing (uses two hash functions)

Separate Chaining

- The hash table is implemented as an array of linked lists.

operation	Index	Comments
Inserting an item, r	hash address, i	insertion into the linked list at position i. Synonyms are chained in the same linked list
Retrieval of an item,	hash address, i	retrieval from the linked list at position i
Deletion of an item, r	hash address, i	deleting r from the linked list at position i.



Separate Chaining...

- Example: Load the keys 23, 13, 21, 14, 7, 8, and 15 , in this order, in a hash table of size 7 using separate chaining with the hash function: h(key) = key % 7

h(23) = 23 % 7 = 2

h(13) = 13 % 7 = 6

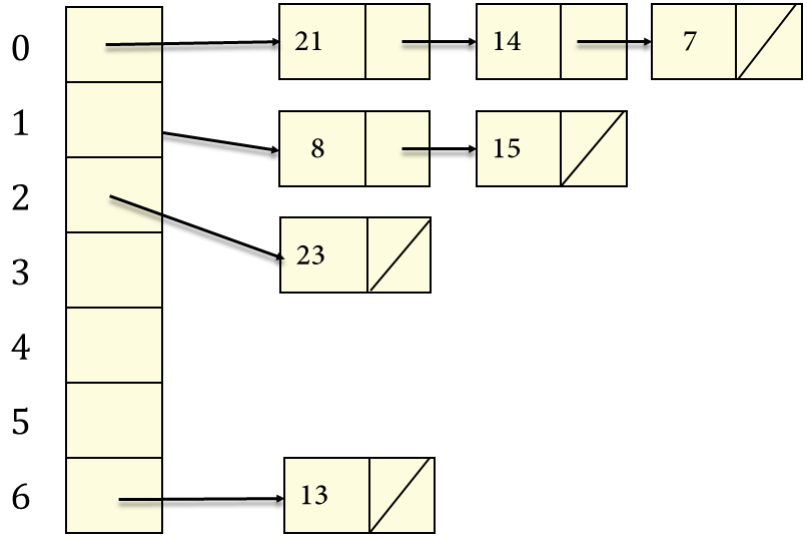
h(21) = 21 % 7 = 0

h(14) = 14 % 7 = 0 collision

h(7) = 7 % 7 = 0 collision

h(8) = 8 % 7 = 1

h(15) = 15 % 7 = 1 collision



Separate Chaining with String Keys

- Common ways to hash a string:
  - Character sum
    - A hash function for a string $s = c_0c_1c_2\ldots c_{n-1}$ be defined as:
      - $h = (c_0 + c_1 + c_2 + \ldots + c_{n-1}) \% tableSize$

```
public static int hash(String key, int tableSize){
  int hashValue = 0;
  for (int i = 0; i < key.length(); i++){
        hashValue += key.charAt(i);
  }
  return hashValue % tableSize;
```

  - Polynomial Hash
    - Treat the string as a **polynomial**, where each character is a coefficient
    - Produces few collisions and distributes strings uniformly

$$h = \left[\sum_{k=0}^{s.length()-1} c_k \cdot p^k \right] \% \, tableSize$$

  - Alternative hash functions

$$h = (c_0 + 27c_1 + 729c_2) \% \, tableSize$$

$$h = (c_0 + c_{n-1} + s.length()) \% \, tableSize$$

Separate Chaining with String Keys...

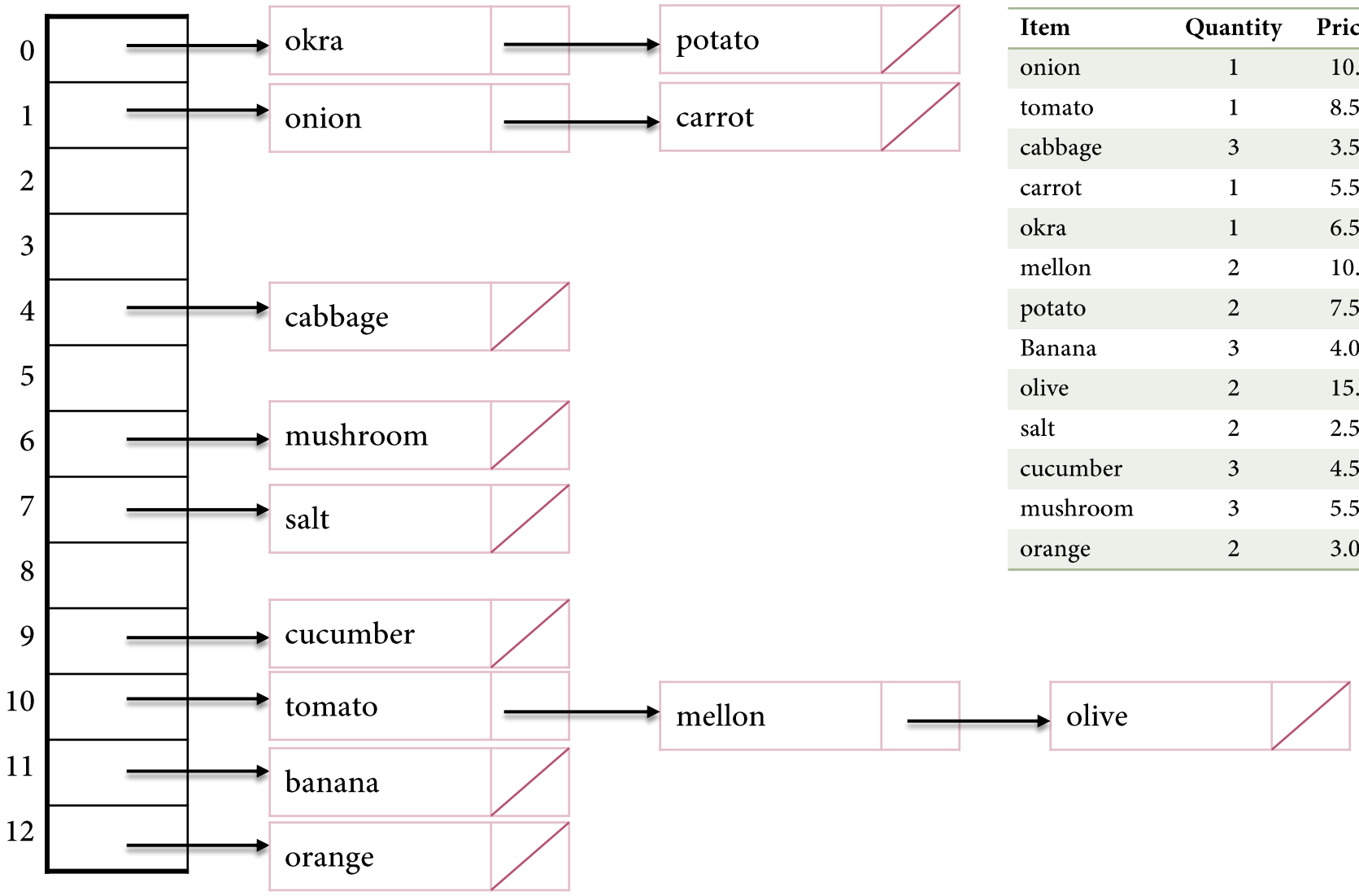
- Use the hash function **hash** to load the following commodity items into a hash table of size 13 using separate chaining:

Item	Quantity	Price
onion	1	10.0
tomato	1	8.50
cabbage	3	3.50
carrot	1	5.50
okra	1	6.50
mellon	2	10.0
potato	2	7.50
Banana	3	4.00
olive	2	15.0
salt	2	2.50
cucumber	3	4.50
mushroom	3	5.50
orange	2	3.00

Character	ASCII code
a	97
b	98
c	99
e	101
g	103
h	104
i	105
k	107
l	108
m	109
n	110
o	111
p	112
r	114
s	115
t	116
u	117
v	118

- Solution:
  - hash(onion) = (111 + 110 + 105 + 111 + 110) % 13 = 547 % 13 = 1
  - hash(salt) = (115 + 97 + 108 + 116) % 13 = 436 % 13 = 7
  - hash(orange) = (111 + 114 + 97 + 110 + 103 + 101)%13 = 636 %13 = 12

Separate Chaining with String Keys...



Item	Quantity	Price	H(Key)
onion	1	10.0	1
tomato	1	8.50	10
cabbage	3	3.50	4
carrot	1	5.50	1
okra	1	6.50	0
mellon	2	10.0	10
potato	2	7.50	0
Banana	3	4.00	11
olive	2	15.0	10
salt	2	2.50	7
cucumber	3	4.50	9
mushroom	3	5.50	6
orange	2	3.00	12

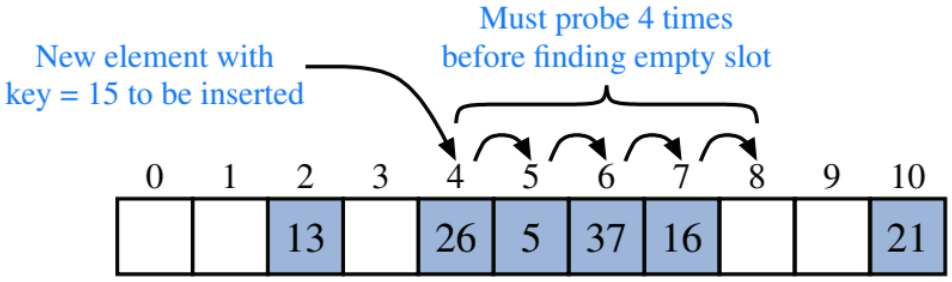
Introduction to Open Addressing

- A type of collision resolution strategy that resolves collisions by choosing a different location when the natural choice is full.
- All items are stored in the hash table itself.
  - In addition to the cell data (if any), each cell keeps one of the three states: EMPTY, OCCUPIED, DELETED.
- While inserting, if a collision occurs, alternative cells are tried until an empty cell is found.
- Deletion: (lazy deletion): When a key is deleted the slot is marked as DELETED rather than EMPTY otherwise subsequent searches that hash at the deleted cell will fail.
- Probe sequence: A probe sequence is the sequence of array indexes that is followed in searching for an empty cell during an insertion, or in searching for a key during find or delete operations.
- The most common probe sequences are of the form:
  - hi(key) = [h(key) + c(i)] % n, for i = 0, 1, …, n-1. where h is a hash function and n is the size of the hash table.
- The function c(i) is used to resolve collisions and must have the following properties:
  - Property 1: c(0) = 0
  - Property 2: The set of values {c(0) % n, c(1) % n, c(2) % n, . . . , c(n-1) % n} must be a permutation of {0, 1, 2,. . ., n – 1}, that is, it must contain every integer between 0 and n - 1 inclusive.

Introduction to Open Addressing...

- To insert item r, we examine array location $h_0(r) = h(r)$.
  - If there is a collision, array locations $h_1(r), h_2(r), ..., h_{n-1}(r)$ are examined until an empty or deleted slot is found. the key is inserted into that cell.

- Similarly, to find item r, we examin the same sequence of locations in the same order.



- Note: For a given hash function h(key), the only difference in the open addressing collision resolution techniques (linear probing, quadratic probing and double hashing) is in the definition of the function c(i).
- Common definitions of c(i) are:

Collision resolution technique	c(i)
Linear probing	i
Quadratic probing	$\pm i^2$
Double hashing	$i * h_p(key)$

where $\mathbf{h_p(key)}$ is another hash function.

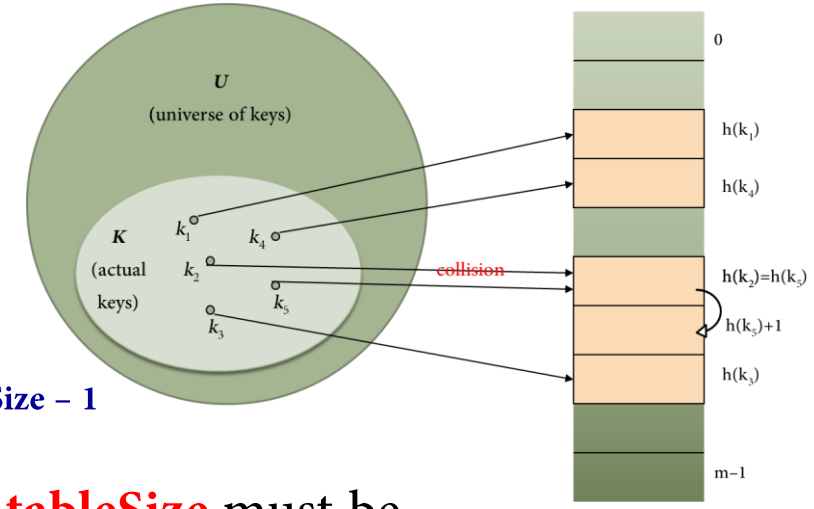
Open Addressing: Linear Probing

- **c(i)** is a linear function in **i** of the form **c(i) = a*i.**
- Usually **c(i)** is chosen as:

  c(i) = i for i = 0, 1, . . . , tableSize – 1

- The probe sequences are then given by:

  h_i(key) = [h(key) + i] % tableSize for i = 0, 1, . . . , tableSize – 1



- For **c(i) = a*i** to satisfy Property 2, **a** and **tableSize** must be relatively prime.

```
LinearProbingInsert(k)
    if (table is full) then error
    probe := h(k)
    while (table[probe] occupied) do
        probe := (probe+1) mod m
    table[probe] = k
```

If h(key) is already occupied,

- try (h(key) + 1) % TableSize. If occupied,
- try (h(key) + 2) % TableSize. If occupied,
- try (h(key) + 3) % TableSize. If occupied

and so on

Linear Probing Example

- Example: Perform the operations given below, in the given order, on an initially empty hash table of size 13 using linear probing with c(i) = i and the hash function: h(key) = key % 13:

  insert(18), insert(26), insert(35), insert(9), find(15), find(48), delete(35), delete(40), find(9), insert(64), insert(47), find(35)

- The required probe sequences are given by:

  hi(key) = (h(key) + i) % 13 i = 0, 1, 2, . . ., 12

Linear Probing Example...

Index	Status	Value
0	E	
1	E	
2	E	
3	E	
4	E	
5	E	
6	E	
7	E	
8	E	
9	E	
10	E	
11	E	
12	E	

Initial state of the hash table:

Status: E for Empty, O for Occupied, D for Deleted

h(key) = key % 13

Index	Status	Value
0	E	
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	E	
10	E	
11	E	
12	E	

insert(18)

h(18) = 18 % 13 = 5

~~h_i(key) = (h(key) + i) % 13~~

~~h_0(18) = (5 + 0) % 13 = 5~~

cell 5 has EMPTY (E) status, 18 is inserted at cell 5 and status is changed to Occupied (O).

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	E	
10	E	
11	E	
12	E	

insert(26)

h(26) = 26 % 13 = 0

~~h_i(key) = (h(key) + i) % 13~~

~~h_0(26) = (0 + 0) % 13 = 0~~

cell 0 has EMPTY (E) status, 26 is inserted at cell 0 and status is changed to Occupied (O).

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	O	35
10	E	
11	E	
12	E	

insert(35)

h(35) = 35 % 13 = 9

~~h_i(key) = (h(key) + i) % 13~~

~~h_0(35) = (9 + 0) % 13 =9~~

cell 9 has EMPTY (E) status, 35 is inserted at cell 9 and status is changed to Occupied (O).

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	O	35
10	O	9
11	E	
12	E	

insert(9)

h(9) = 9 % 13 = 9

h_i(key) = (h(key) + i) % 13

h_0(9) = (9 + 0) % 13 = 9

COLLISION

h_1(9) = (9 + 1) % 13 = 10

cell 10 has EMPTY (E) status, 9 is inserted at cell 10 and status is changed to Occupied (O).

Linear Probing Example...

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	O	35
10	O	9
11	E	
12	E	

find(15)

h(15) = 15 % 13 = 2

~~hi(key) = (h(key) + i) % 13~~

~~h0(15) = (2 + 0) % 13 = 2~~

search fails because cell 2 has EMPTY (E) Status

h(key) = key % 13

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	D	35
10	O	9
11	E	
12	E	

find(48)

h(48) = 48 % 13 = 9

hi(key) = (h(key) + i) % 13

h0(48) = (9 + 0) % 13 = 9

continue search, because cell 9 has OCUPIED (O) status not 48

h1(48) = (9 + 1) % 13 = 10

continue search, because cell 10 has OCUPIED (O) status not 48

h2(48) = (9 + 2) % 13 = 11

search fails because cell 11 has EMPTY (E) Status

delete(35)

h(35) = 35 % 13 = 9

hi(key) = (h(key) + i) % 13

h0(35) = (9 + 0) % 13 = 9

35 found and deleted, cell 9 status changed to DELETED (D)

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	D	35
10	O	9
11	E	
12	E	

insert(9)

h(9) = 9 % 13 = 9

hi(key) = (h(key) + i) % 13

h0(9) = (9 + 0) % 13 = 9

cell 9 has DELETED (D) status. CONTINUE SEARCHING.

h1(9) = (9 + 1) % 13 = 10

10 has OCUPIED with 9 ⟶ Duplicate key. Cannot insert 9

Linear Probing Example...

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	D	35
10	O	9
11	E	
12	E	

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	D	35
10	O	9
11	E	
12	E	

delete(40)

h(40) = 40 % 13 = 1

hi(key) = (h(key) + i) % 13

h0(40) = (1 + 0) % 13 = 1

cell 1 has EMPTY (E) status. 40 not found, deletion FAILS.

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	E	
9	D	35
10	O	9
11	E	
12	O	64

insert(64)

h(64) = 64 % 13 = 12

~~hi(key) = (h(key) + i) % 13~~

~~h0(64) = (12 + 0) % 13 = 12~~

cell 12 has EMPTY (E) status, 64 is inserted at cell 12 and status is changed to Occupied (O).

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	O	47
9	D	35
10	O	9
11	E	
12	O	64

insert(47)

h(47) = 47 % 13 = 8

~~hi(key) = (h(key) + i) % 13~~

~~h0(47) = (8 + 0) % 13 = 8~~

cell 8 has EMPTY (E) status, 47 is inserted at cell 8 and status is changed to Occupied (O).

find(35)

h(35) = 35 % 13 = 9

hi(key) = (h(key) + i) % 13

h0(35) = (9 + 0) % 13 = 9

cell 9 has Deleted (D) status, search continues.

h1(35) = (9 + 1) % 13 = 10

cell 10 has Occupied (O) status, not 35, search continues.

h2(35) = (9 + 2) % 13 = 11

cell 11 has Empty (E) status, search FAILS.

h(key) = key % 13

Linear Probing Example...

Index	Status	Value
0	O	26
1	E	
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	O	47
9	D	35
10	O	9
11	E	
12	O	64

h(key) = key % 13

Index	Status	Value
0	O	26
1	O	77
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	O	47
9	D	35
10	O	9
11	E	
12	O	64

insert(77)
h(77) = 77 % 13 = 12
hi(key) = (h(key) + i) % 13
h0(77) = (12 + 0) % 13 = 12
cell 12 has Occupied (O) status, COLLISSION.
h1(77) = (12 + 1) % 13 = 0
cell 0 has Occupied (O) status, COLLISSION.
h2(77) = (12 + 2) % 13 = 1
cell 1 has Empty (E) status, 77 is inserted at cell 1.

Index	Status	Value
0	O	26
1	O	77
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	O	47
9	O	21
10	O	9
11	E	
12	O	64

insert(21)
h(21) = 21 % 13 = 8
hi(key) = (h(key) + i) % 13
h0(21) = (8 + 0) % 13 = 8
cell 8 has Occupied (O) status, COLLISSION.
h1(21) = (8 + 1) % 13 = 9
cell 9 has Deleted (D) status, continue.
h2(21) = (8 + 2) % 13 = 10
cell 10 has Occupied (O) status, with different value continue.
h3(21) = (8 + 3) % 13 = 11
cell 11 has Empty (E) status, 21 is inserted in cell 9 and status is changed to Occupied (O).

Index	Status	Value
0	D	26
1	O	77
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	O	47
9	O	21
10	O	9
11	E	
12	O	64

Delete(26)
h(26) = 26 % 13 = 0
hi(key) = (h(key) + i) % 13
h0(26) = (0 + 0) % 13 = 0
Cell 0 status is changed to Deleted (D) status.

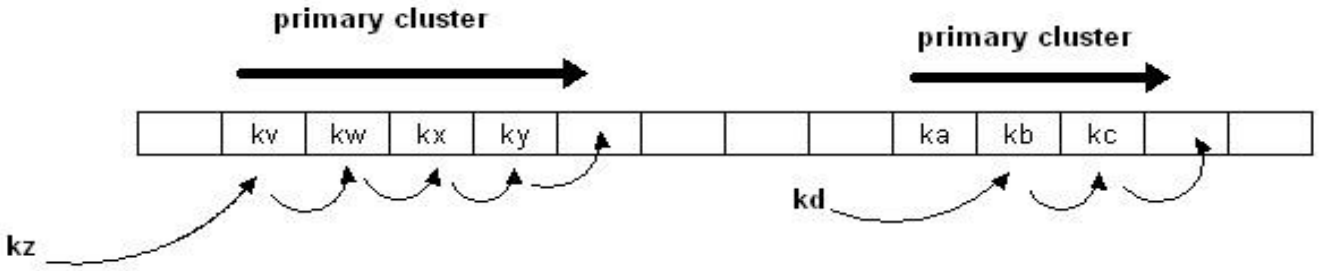
Index	Status	Value
0	O	39
1	O	77
2	E	
3	E	
4	E	
5	O	18
6	E	
7	E	
8	O	47
9	O	21
10	O	9
11	E	
12	O	64

insert(39)
h(39) = 39 % 13 = 0
hi(key) = (h(key) + i) % 13
h0(39) = (0 + 0) % 13 = 0
Cell 0 status is Deleted (D), continue.
h1(39) = (0 + 1) % 13 = 1
Cell 1 status is Occupied (O) with different value.
h2(39) = (0 + 2) % 13 = 2
Cell 2 status is Empty (E), hence 39 is inserted in cell 0 and the status is changed to Occupied (O).

Disadvantage of Linear Probing: Primary Clustering

- Linear probing is subject to a primary clustering phenomenon.
  - Elements tend to cluster around table locations that they originally hash to.
- Primary clusters can combine to form larger clusters.
  - This leads to long probe sequences and hence deterioration in hash table efficiency.
  - Small clusters tend to form large clusters

long chains of occupied slots within a hash table



Example of a primary cluster: Insert keys: **18, 41, 22, 44, 59, 32, 31, 73**, in this order, in an originally empty hash table of size **13**, using the hash function **h(key) = key % 13** and **c(i) = i**:

h(18) = 5

h(41) = 2

h(22) = 9

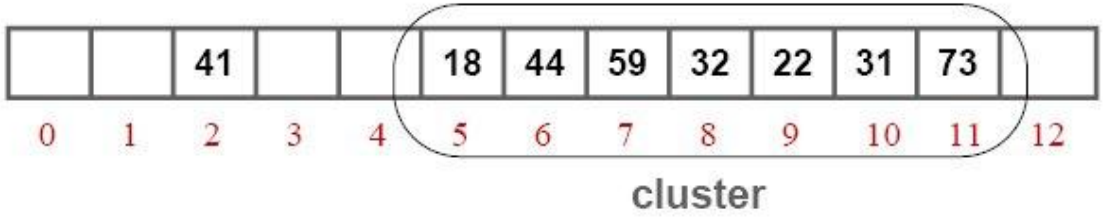
h(44) = 5+1

h(59) = 7

h(32) = 6+1+1

h(31) = 5+1+1+1+1+1

h(73) = 8+1+1+1a



Open Addressing: Quadratic Probing

- Quadratic probing eliminates primary clusters
  - c(i) is a quadratic function in i of the form $c(i) = a * i^2 + b * i$ (quadratic with linear propping)
  - Usually c(i) is chosen as:

    $c(i) = \pm i^2$ for i = 0, 1, . . ., (tableSize – 1) / 2 (pure quadratic)
- The probe sequences are then given by:
- $hi(key) = [h(key) \pm i^2]$ % tableSize for i = 0, 1, . . ., (tableSize – 1)/ 2
- Note for Quadratic Probing:
  - Hashtable size should not be an even number; otherwise, Property 2* will not be satisfied
  - Ideally, table size should be a prime of the form 4j + 3, where j is an integer. This choice of table size guarantees Property 2*.
- * Property 2 of the function used in probe sequence

Open Addressing: Quadratic Probing

- Quadratic probing can cycle through the same full indices, never terminating
- despite table not being ful
  - For $\mathbf{h_0(key) = [h(key) \pm i^2]}$ % **tableSize** the probe sequence starts at $\mathbf{h_0(key) = h(key)}$ and then it examines cells 1, -1, 4, -4, 9, -9 and so on, away from the original probe. For this probe sequence, normalization is done when a **negative** index is computed:

    normalizedIndex = (computedIndex + tableSize) % tableSize

- Example: Load the keys **23, 13, 21, 14, 7, 8, and 15**, in this order, in a hash table of size 7 using quadratic probing with $\mathbf{c(i) = \pm i^2}$ and the hash function: **h(key) = key % 7**
- The required probe sequences are given by:

$$h_i(key) = (h(key) \pm i^2) \% 7 \quad i = 0, 1, 2, 3$$

If h(key) is already occupied,

- try (h(key) + 1) % TableSize. If occupied,
- try (h(key) - 1) % TableSize. If occupied,
- try (h(key) + 4) % TableSize. If occupied
- try (h(key) - 4) % TableSize. If occupied

and so on

Quadratic Probing...

$h_i(key) = (h(key) \pm i^2) \% 7 \quad i = 0, 1, 2, 3$

Load the keys 23, 13, 21, 14, 7, 8, and 15

$h_0(23) = (23 \% 7) \% 7 = 2$

$h_0(13) = (13 \% 7) \% 7 = 6$

$h_0(21) = (21 \% 7) \% 7 = 0$

$h_0(14) = (14 \% 7) \% 7 = 0$ collision

$h_1(14) = (0 + 1^2) \% 7 = 1$

$h_0(7) = (7 \% 7) \% 7 = 0$ collision

$h_1(7) = (0 + 1^2) \% 7 = 1$ collision

$h_{-1}(7) = (0 - 1^2) \% 7 = -1$

NORMALIZE: $(-1 + 7) \% 7 = 6$ collision

$h_2(7) = (0 + 2^2) \% 7 = 4$

$h_0(8) = (8 \% 7) \% 7 = 1$ collision

$h_1(8) = (1 + 1^2) \% 7 = 2$ collision

$h_{-1}(8) = (1 - 1^2) \% 7 = 0$ collision

$h_2(8) = (1 + 2^2) \% 7 = 5$

$h_0(15) = (15 \% 7) \% 7 = 1$ collision

$h_1(15) = (1 + 1^2) \% 7 = 2$ collision

$h_{-1}(15) = (1 - 1^2) \% 7 = 0$ collision

$h_2(15) = (1 + 2^2) \% 7 = 5$ collision

$h_{-2}(15) = (1 - 2^2) \% 7 = -3$

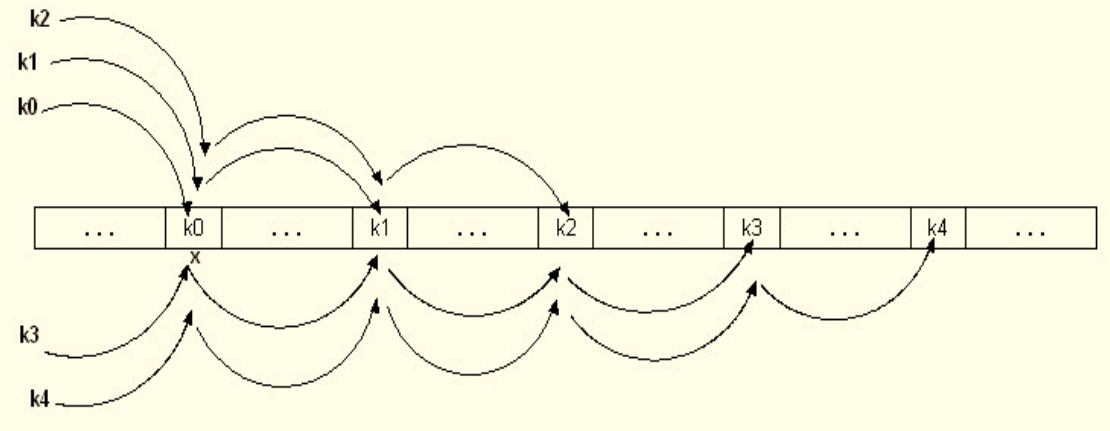
NORMALIZE: $(-3 + 7) \% 7 = 4$ collision

$h_3(15) = (1 + 3^2) \% 7 = 3$

0	O	21
1	O	14
2	O	23
3	O	15
4	O	7
5	O	8
6	O	13

Secondary Clusters

- Quadratic probing is better than linear probing because it eliminates primary clustering.
- However, it may result in secondary clustering: if h(k1) = h(k2) the probing sequences for k1 and k2 are exactly the same. This sequence of locations is called a secondary cluster.
- Secondary clustering is less harmful than primary clustering because secondary clusters do not combine to form large clusters.
- Example of Secondary Clustering:
  - Suppose keys k_0, k_1, k_2, k_3, and k_4 are inserted in the given order in an originally empty hash table using quadratic probing with $c(i) = i^2$.
  - Assuming that each of the keys hashes to the same array index x. A secondary cluster will develop and grow in size:



Double Hashing

- To eliminate secondary clustering, synonyms must have different probe sequences.
- Double hashing achieves this by having two hash functions that both depend on the hash key.
- c(i) = i * hp(key) for i = 0, 1, . . . , tableSize – 1

  where hp (or h2) is another hash function.

- The probing sequence is:
- $h_i(key) = [h(key) + i*hp(key)]\%\ tableSize$ for i = 0, 1, . . . , tableSize – 1

- The function c(i) = i*hp(r) satisfies Property 2 provided hp(r) and tableSize are relatively prime.
- To guarantee Property 2, tableSize must be a prime number.
- Common definitions for hp are :
  - hp(key) = 1 + key % (tableSize - 1)
  - hp(key) = q - (key % q) where q is a prime less than tableSize
  - hp(key) = q *(key % q) where q is a prime less than tableSize

Double Hashing...

- Performance of Double hashing:
  - Much better than linear or quadratic probing because it eliminates both primary and secondary clustering.
  - BUT requires a computation of a second hash function hp.
- Example:
  - Use double hashing to load the keys 18, 26, 35, 9, 64, 47, 96, 36, 70, 12 in this order, in an empty hash table of size 13. Then perform find(7).
  - Use the first hash function: h(key) = key % 13 and
  - the second hash function: hp(key) = 1 + key % 12

```
DoubleHashingInsert(k)
    if (table is full) then error
    probe := h1(k)
    offset := h2(k)
    while (table[probe] occupied) do
        probe := (probe + offset) mod m
    table[probe] := k
```

If h(key) is already occupied,

Compute h_p(key)

- try (h(key) + h_p(key) % TableSize. If occupied,
- try (h(key) + 2 h_p(key)) % TableSize. If occupied,
- try (h(key) + 3 * h_p(key)) % TableSize. If occupied

and so on

Double Hashing...

E	E	E	E	E	E	E	E	E	E	E	E	E
0	1	2	3	4	5	6	7	8	9	10	11	12

E	E	E	E	E	O	E	E	E	E	E	E	E
0	1	2	3	4	5	6	7	8	9	10	11	12
					18							

O	E	E	E	E	O	E	E	E	E	E	E	E
0	1	2	3	4	5	6	7	8	9	10	11	12
26					18							

O	E	E	E	E	O	E	E	E	O	E	E	E
0	1	2	3	4	5	6	7	8	9	10	11	12
26					18				35			

O	E	E	E	E	O	O	E	E	O	E	E	E
0	1	2	3	4	5	6	7	8	9	10	11	12
26					18	9			35			

$h_i(key) = [h(key) + i*h_p(key)]\% 13$

$h(key) = key \% 13$

$h_p(key) = 1 + key \% 12$

insert(18)

h(18) = 18 % 13 = 5

$h_0(18) = (5 + 0) \% 13 = 5$

insert(26)

h(26) = 26 % 13 = 0

$h_0(26) = (0 + 0) \% 13 = 0$

insert(35)

h(35) = 35 % 13 = 9

$h_0(35) = (9 + 0) \% 13 = 9$

insert(9)

h(9) = 9 % 13 = 9 collision

hp(9) = 1 + 9 % 12 = 10

$h_0(9) = (9 + 0) \% 13 = 9$

$h_1(9) = (9 + 1 * 10) \% 13 = 6$

Double Hashing...

O	E	E	E	E	O	O	E	E	O	E	E	E
0	1	2	3	4	5	6	7	8	9	10	11	12
26					18	9			35			

O	E	E	E	E	O	O	E	E	O	E	E	O
0	1	2	3	4	5	6	7	8	9	10	11	12
26					18	9			35			64

O	E	E	E	E	O	O	E	O	O	E	E	O
0	1	2	3	4	5	6	7	8	9	10	11	12
26					18	9		47	35			64

O	E	E	E	E	O	O	O	O	O	E	E	O
0	1	2	3	4	5	6	7	8	9	10	11	12
26					18	9	96	47	35			64

O	E	E	E	E	O	O	O	O	O	O	E	O
0	1	2	3	4	5	6	7	8	9	10	11	12
26					18	9	96	47	35	36		64

$h_i(key) = [h(key) + i*h_p(key)]\% 13$

$h(key) = key \% 13$

$h_p(key) = 1 + key \% 12$

insert(64)

h(64) = 64 % 13 = 12

$h_0(64) = (12 + 0) \% 13 = 12$

insert(47)

h(47) = 47 % 13 = 8

$h_0(47) = (8 + 0) \% 13 = 8$

insert(96)

h(96) = 96 % 13 = 5 collision

hp(96) = 1 + 96 % 12 = 1

$h_0(96) = (5 + 0) \% 13 = 5$ collision

$h_1(96) = (5 + 1 * 1) \% 13 = 6$ collision

$h_2(96) = (5 + 2 * 1) \% 13 = 7$

insert(36)

h(36) = 36 % 13 = 10

$h_0(36) = (10 + 0) \% 13 = 10$

Double Hashing...

O	E	E	E	E	O	O	O	O	O	O	E	O
0	1	2	3	4	5	6	7	8	9	10	11	12
26					18	9	96	47	35	36		64

O	E	E	O	E	O	O	O	O	O	O	E	O
0	1	2	3	4	5	6	7	8	9	10	11	12
26			70		18	9	96	47	35	36		64

O	O	E	O	E	O	O	O	O	O	O	E	O
0	1	2	3	4	5	6	7	8	9	10	11	12
26	12		70		18	9	96	47	35	36		64

O	O	E	O	E	O	O	O	O	O	O	E	O
0	1	2	3	4	5	6	7	8	9	10	11	12
26	12		70		18	9	96	47	35	36		64

$h_i(key) = [h(key) + i*h_p(key)]\% 13$

$h(key) = key \% 13$

$h_p(key) = 1 + key \% 12$

insert(70)

h(70) = 70 % 13 = 5 collision

hp(70) = 1 + 70 % 12 = 11

$h_0(70) = (5 + 0) \% 13 = 5$ collision

$h_1(70) = (5 + 1 * 11) \% 13 = 3$

insert(12)

h(12) = 12 % 13 = 12

hp(12) = 1 + 12 % 12 = 1

$h_0(12) = (12 + 0) \% 13 = 12$ collision

$h_1(12) = (12 + 1 * 1) \% 13 = 0$ collision

$h_2(12) = (12 + 2 * 1) \% 13 = 1$

find(7)

h(7) = 7 % 13 = 7 occupied

hp(7) = 1 + 7 % 12 = 8

$h_0(7) = (7 + 0) \% 13 = 7$ occupied

$h_1(7) = (7 + 1 * 8) \% 13 = 2$ empty

Search FAILS because slot 2 has status EMPTY

Double Hashing...

O	O	E	O	E	O	O	O	O	O	O	E	O
0	1	2	3	4	5	6	7	8	9	10	11	12
26	12		70		18	9	96	47	35	36		64

$h_i(key) = [h(key) + i*h_p(key)]\% 13$

$h(key) = key \% 13$

$h_p(key) = 1 + key \% 12$

$h_0(18) = (5 + 0) \% 13 = 5$

$h_0(26) = (0 + 0) \% 13 = 0$

$h_0(35) = (9 + 0) \% 13 = 9$

$h_0(9) = (9 + 0) \% 13 = 9$ collision

$h_p(9) = 1 + 9 \% 12 = 10$

$h_1(9) = (9 + 1 * 10) \% 13 = 6$

$h_0(64) = (12 + 0) \% 13 = 12$

$h_0(47) = (8 + 0) \% 13 = 8$

$h_0(96) = (5 + 0) \% 13 = 5$ collision

$h_p(96) = 1 + 96 \% 12 = 1$

$h_1(96) = (5 + 1 * 1) \% 13 = 6$ collision

$h_2(96) = (5 + 2 * 1) \% 13 = 7$

$h_0(36) = (10 + 0) \% 13 = 10$

$h_0(70) = (5 + 0) \% 13 = 5$ collision

$h_p(70) = 1 + 70 \% 12 = 11$

$h_1(70) = (5 + 1 * 11) \% 13 = 3$

$h_0(12) = (12 + 0) \% 13 = 12$ collision

$h_p(12) = 1 + 12 \% 12 = 1$

$h_1(12) = (12 + 1 * 1) \% 13 = 0$ collision

$h_2(12) = (12 + 2 * 1) \% 13 = 1$

$h_0(7) = (7 + 0) \% 13 = 7$ occupied

$h_p(7) = 1 + 7 \% 12 = 8$

$h_1(7) = (7 + 1 * 8) \% 13 = 2$ empty

Search Fails because slot 2 has status Empty

Load Factor

- Load Factor (is a measure of how full the hash table is.
  - It is defined as $\lambda = \frac{\text{number of items}}{\text{table size}}$.

- Example:
  - Suppose your hash table size is **10**, and you stored **6** elements: $\lambda = \frac{6}{10}$

- Why it matters?
  - The load factor affects **performance**:
  - Low load factor: Fewer collisions $\longrightarrow$ Faster operations (e.g. insert)
  - High load factor: More collisions $\longrightarrow$ Slower performance

- With open addressing, the load factor cannot exceed 1.
- With chaining, the load factor often exceeds 1.

Open Addressing Facts

- In general, primes give the best table sizes.
- With any open addressing method of collision resolution, as the table fills, there can be a severe degradation in the table performance.
- Load factors between 0.6 and 0.7 are common.
- Load factors > 0.7 are undesirable.
- The search time depends only on the load factor, not on the table size.
- We can use the desired load factor to determine appropriate table size:

$$\text{table size} = \text{smallest prime} \geq \frac{\text{number of items in table}}{\text{desired load factor}}$$

Rehashing

- As noted before, with open addressing, if a hash table becomes too full, performance can suffer a lot.
- So, what can we do if we want to insert more keys?
- We can:
  - double the hash table size by using a prime number that is approximately twice as large as the current table size,
  - use this number to modify the hash function, and
  - rehash all values stored in the original table to the new table, and then
  - hash each of the new keys to the new table.
- What is the cost of rehashing?
  - Rehashing takes O(n)
  - However, it occurs infrequently.

Rehashing Example

0	1	2	3	4	5	6
6	15		24			13

$\lambda = \frac{4}{7} = 0.57$

0	1	2	3	4	5	6
6	15	23	24			13

Insert 23 $\lambda = \frac{5}{7} = 0.71$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
						6	23	24					13		15	

rehashe

h(x) = x % 17

$\lambda = \frac{5}{17} = 0.29$

h(6) = 6 % 17 = 6

h(15) = 15 % 17 = 15

collision

h(23) = 23 % 17 = 6

collision

h(23) = (6 + 1) % 17 = 7

h(24) = 24 % 17 = 7

h(24) = (7 + 1) % 17 = 8

h(13) = 13 % 17 = 13

Scan down the entire original hashtable, recompute new hash value for each non-deleted key and inserting it in the new table.

When rehashing with double hashing, the hash function or functions that depend on the table size are the ones that need to be modified.

Rehashing to remove cells with DELETED status

- After many deletions, a hash table may become clogged with DELETED-status cells. These cells must still be scanned as part of a cluster. It may be beneficial to periodically rehash all valid items.
- Example with linear probing and h(key) = key % 23
  - h(75) = 75 % 23 = 6

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
	X	X				X	29	X	X	X	X	54	X	60	X	X	35	X	X		21	X

$\lambda = \frac{5}{23} \approx 0.217$

Search(75): h(75) = 75 % 23 = 6 requires 15 probes

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
						29		54				35		60							21	

$\lambda = \frac{5}{23} \approx 0.217$

Search(75) requires 2 probes

Note: In this case, the same hash function and the same table size are used in rehashing.

Introduction to Open Addressing...

- Advantages of Open addressing:
  - All items are stored in the hash table itself. There is no need for another data structure.
  - Open addressing is more efficient storage-wise.

- Disadvantages of Open Addressing:
  - The keys of the objects to be hashed must be distinct.
  - Dependent on choosing a proper table size.
  - Requires the use of a three-state (Occupied, Empty, or Deleted) flag in each cell.

Separate Chaining versus Open-addressing

- Separate Chaining has several advantages over open addressing:
  - Collision resolution is simple and efficient.
  - The hash table can hold more elements without the large performance deterioration of open addressing (The load factor can be 1 or greater)
  - The performance of chaining declines much more slowly than open addressing.
  - Deletion is easy - no special flag values are necessary.
  - Table size need not be a prime number.
  - The keys of the objects to be hashed need not be unique.

- Disadvantages of Separate Chaining:
  - It requires the implementation of a separate data structure for chains, and code to manage it.
  - The main cost of chaining is the extra space required for the linked lists.
  - For some languages, creating new nodes (for linked lists) is expensive and slows down the system.

Some Applications of Hash Tables

Database systems	Fast retrieval of records using primary keys
Compiler Design	Implementing symbol tables to store variables and functions.
Data dictionaries	Mapping keys to values in programming languages
Network Applications	Route lookup, packet classification, and network monitoring.
Caching	Storing frequently accessed data (e.g., web caches, CPU caches).
Password Storage	Securely storing hashed passwords for authentication.

Problems for Which Hash Tables are not Suitable

- Hash tables are efficient for exact key-based lookups, but they are not suitable for several types of problems due to their unordered nature and design limitations:

Range Queries	Hash tables cannot efficiently find all keys within a given range (e.g., values between 10 and 50) because keys are not stored.
Prefix or Pattern Matching	Hash tables are unsuitable for queries like "all keys starting with a prefix"
No unique keys	Open-addressed hash tables cannot be used if the data does not have unique keys. An alternative is using separate-chained hash tables.